How to Run the Phi-3.5 RAG CLI Demo (Windows)

Roland Vaatmann

December 26, 2025

1 Overview

This repository contains a command-line (CLI) Retrieval-Augmented Generation (RAG) demo that runs **Phi-3.5-mini-instruct** locally/on-device using **ONNX Runtime GenAI**. The app ingests documents (PDF/TXT/MD), builds a vector index, retrieves relevant chunks for a user question, and generates an answer grounded in the retrieved context.

Two inference accelerators are supported:

- **CPU** (default)
- **NVIDIA CUDA GPU** (enabled with `-provider cuda`)

2 What is included in the ZIP

The submission ZIP includes:

- Source code under `src/`
- Pre-created Python virtual environments: `.venv_cpu/` and `.venv_cuda/`
- The RAG index output folder path convention: `data/index/` (created after ingest)

Note: Models may be included or may need to be downloaded separately depending on ZIP size constraints. The expected model directory structure is documented in Section 4.

3 Prerequisites (Machine Setup)

- Windows 11 (x64)
- NVIDIA GPU driver installed (for CUDA runs)
- CUDA Toolkit 12.5 installed (for CUDA runs) Here
- NVIDIA cuDNN Here
- Python 3.11.9
- Only needed if recreating env Microsoft C++ Build Tools (for hnswlib install)

3.1 PowerShell Execution Policy

If activation scripts are blocked, run PowerShell as the current user:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

3.2 Path Length Note

To avoid Windows path-length issues, extract the ZIP into a short path such as:

```
C:\work\rag-phi35\
```

4 Model Files

The demo expects Phi-3.5-mini-instruct ONNX INT4 AWQ variants in `models/`.
Expected folders:

- CPU model: `models\Phi-3.5-mini-instruct-onnx\cpu_and_mobile\cpu-int4-awq-block-128-acc-lev`
- GPU model: `models\Phi-3.5-mini-instruct-onnx\gpu\gpu-int4-awq-block-128`

5 Running the Demo

All commands should be run from the repository root folder (the folder containing `src/`).

5.1 1) Ingest documents (build the vector index)

Place your knowledge base documents into `kb/`. For example, put a PDF into:

```
kb\nasa_systems_engineering_handbook.pdf
```

Activate the CPU environment and run ingest:

```
cd <PATH_TO_REPO_ROOT>
.\.venv_cpu\Scripts\Activate.ps1
python -m src.cli ingest --docs-dir kb --out-dir data/index
```

5.2 2) Ask a question on CPU

CPU is the default provider:

```
.\.venv_cpu\Scripts\Activate.ps1
python -m src.cli ask "What is the difference between product verification and product
    validation?"
```

5.3 3) Ask a question on CUDA GPU

Activate the CUDA environment and use `-provider cuda`:

```
.\.venv_cuda\Scripts\Activate.ps1
python -m src.cli ask "What is the difference between product verification and product
    validation?" --provider cuda
```

5.4 CLI Output Metrics

For each query, the CLI prints:

- `device_type` (CPU or CUDA) and `model_load_s`
- `prompt_tokens`, `new_tokens`
- `ttft_s`, `gen_total_s`, `tok/s`

6 Troubleshooting

6.1 CUDA provider fails to load (missing DLLs)

If you see missing DLL errors (e.g., `cudnn64_9.dll` or `cublasLt64_12.dll`), verify:

- CUDA Toolkit 12.5 and cuDNN are installed
- `CUDA_PATH` points to the correct CUDA install (e.g., `...\CUDA\v12.5`)
- `%CUDA_PATH%\bin` is available for DLL loading